AGENTIKA 2.0 · HUMAN + AI: BETTER TOGETHER · DEADLINE 17 OCT, 12:45 PM PKT

Talkse + FlowCRM Lite: 10-phase build plan

Dono existing repos use karke ek product: **Talkse** phone pe AI receptionist (4 agents), aur **FlowCRM Lite** owner ki screen jahan har call, job aur faisla record hota hai. Niche: **home services** (AC, plumbing, electrical). Demo company: **Cool Breeze AC Services, Islamabad**.

Final setup: 3 services + 1 database

```
Caller (browser mic) ──► TALKSE  (FastAPI, port 8000)  ──► Telegram bot ──► Owner (Ahmed) ka phone
                          Receptionist · Safety · Handoff · Learning
                               │ SQL writes
                               ▼
                     NEON POSTGRES  (schema "public" = FlowCRM, schema "talkse" = Talkse)
                               ▲ reads
Owner laptop ──► FLOWCRM WEB (Next.js, 3000) ──► FLOWCRM API (Express, 4000)   [no login, no AI service]
```

REPO	KYA REUSE HOGA (ALREADY BANA HUA)	KYA NAYA BANEGA
Talkse	`poc.py` : `transcribe` , `extract_intent` , `synthesize` , `merge_state` `booking_engine.py` : slot check, book, reschedule, cancel (idempotent) `clinic_config.py` : service/provider lookup, opening hours `db.py` : appointments table + overlap check	`server.py` (FastAPI), `call_engine.py` (one turn at a time), `static/call.html` , `handoff.py` (Telegram), `learning.py` (FAQ), `crm_sync.py` , `company_data.json` , `Dockerfile`
FlowCRM	Dashboard, Contacts, Leads, Pipeline (drag + lost reason), Tasks, Reports pages; Express API; Prisma schema; Dockerfiles	`AUTH_MODE=demo` switch (Clerk bypass), sidebar filter, seed script, `apps/web/.env.production`

Team split

Person A · Talkse

Voice pipeline, web call page, Telegram handoff, safety and learning agents. Python heavy kaam.

Person B · FlowCRM + data

Database, FlowCRM Lite (demo mode), seed data, `company_data.json` , `crm_sync.py` , deploy checks, README, video prep.

Teen rules jo kabhi nahi todne

1. **Pehle deploy, phir feature.** Phase 1 mein dono apps ka khaali version Alternate Clouds pe live ho. Golden rule: variables set karo → **Rebuild** → **Deploy**. Port code aur dashboard mein same ho, warna 502.
2. **2-hour rule:** agar FlowCRM API + web hour 2 tak live nahi, FlowCRM drop karo aur Person B Talkse mein simple dashboard page banaye.
3. **Feature freeze** deadline se 3 ghante pehle (≈ 9:45 AM). Uske baad sirf bug fix, video, LinkedIn post (khud likho, AI se nahi) aur submission PDF.

Pehle se bana code: Discord #support pe aaj raat poochho ke apna pehle wala code reuse ho sakta hai ya nahi. Dono repos mein naya branch `agentika` banao, pehla commit "Start of Agentika 2.0 work". Submission mein saaf likho: Talkse aur FlowCRM pehle se the; event mein bana: web call, 4-agent handoff, learning loop, CRM integration, demo mode.

Timeline (dono parallel)

HOURS	PERSON A · TALKSE	PERSON B · FLOWCRM + DATA
0 – 1	**Phase 1**: accounts + keys, dono repos ka hello-world deploy	
1 – 2	**Phase 4** start: `call_engine.py`	**Phase 2**: Neon, tables, seed · **Phase 3**: company data
2 – 4	**Phase 4**: `server.py + call.html` (browser booking)	**Phase 3** finish · **Phase 5**: FlowCRM demo mode
4 – 5	**Phase 4** live test	**Phase 5** finish · **Phase 6**: `crm_sync.py`
5 – 7.5	**Phase 7**: Handoff agent + Telegram	**Phase 6** wire-up · 10 test scripts · README
7.5 – 9	**Phase 8**: Safety + Learning agents	Test scripts live pe chalao, bugs report karo
—	**2 ghante rest.** Thaka hua demo points khota hai.	
9 – 10.5	**Phase 9**: final deploy, hardening, full test on mobile data · **feature freeze**	
10.5 – 12	**Phase 10**: video, screenshots, LinkedIn (khud likho), submission PDF · submit by 12:15 PM	

1 Setup aur pehla deploy

1 hour · A · B

- **Keys banao** (kisi repo mein commit nahi): Groq, Deepgram, Neon Postgres, Telegram bot (BotFather se token). Ahmed ka phone bot ko `/start` bheje, phir `getUpdates` se uska `chat_id` note karo.
- **A:** Talkse mein 5-line FastAPI `server.py` (`GET /health`) + `Dockerfile` (python:3.11-slim, `uvicorn server:app --host 0.0.0.0 --port 8000`). Alternate Clouds pe service `talkse`, PORT 8000.
- **B:** FlowCRM API aur web ko jaisa hai waisa deploy karne ki koshish (services `flowcrm-api` port 4000, `flowcrm-web` port 3000). Dockerfiles repo root se build hoti hain (`apps/api/Dockerfile`, `apps/web/Dockerfile`). Agar Alternate Clouds Dockerfile ka path select nahi kar sakta, support se poochho ya root pe copy rakho.
- Discord #support pe pehle wale code ka sawaal. Dono repos mein `agentika` branch.

Done jab: `https://talkse-app.alternatefutures.ai/health` → `{"status":"ok"}`, aur FlowCRM API ka `/health` live. (Web login page pe ruke to theek hai, Phase 5 mein fix hoga.)

2 Database: ek Neon database, do schemas 45 min B

- Neon pe project `coolbreeze`. Do connection strings: pooled → `DATABASE_URL`, direct → `DIRECT_URL` (FlowCRM schema dono maangta hai).
- Laptop se ek dafa: `cd apps/api && npx prisma db push --schema=src/prisma/schema.prisma`. FlowCRM ki tables `public` schema mein ban jayengi. (Dockerfile ka CMD khud db push nahi karta, is liye yeh step zaroori hai.)
- **Talkse ki tables alag schema `talkse` mein:** taake FlowCRM ka `db push` kabhi unhe na chheday. `db.py` ka `init_db()` badlo: `CREATE SCHEMA IF NOT EXISTS talkse`, aur connection pe `SET search_path TO talkse`.

```
talkse.appointments   (existing, db.py se)
talkse.calls          id, caller_phone, caller_name, state_json, status, transcript, crm_contact_id,
crm_lead_id, started_at, ended_at
talkse.escalations    id, call_id, kind (approval|question|hazard), summary, options_json, status
(pending|answered|spoken|timed_out),
                      answer, telegram_message_id, created_at, answered_at
talkse.faqs           id, question, variants_json, answer, source (seed|staff), created_at
```

- **Seed script** (`apps/api/src/prisma/seed.ts`, ya SQL): 1 organization "Cool Breeze AC Services", 1 user "Ahmed (Owner)" + ORG_OWNER membership, 8 contacts, 8 leads, 6 deals mukhtalif stages mein, 4 tasks. Dashboard aur Reports stage pe khaali na lagein. Org id aur user id note karo, env vars mein jayenge.

Done jab: `npx prisma studio` mein seeded rows nazar aayein, aur `talkse.*` chaar tables maujood hon.

3 Niche switch: clinic se home services 1–1.5 hours B A reviews

- `company_data.json` banao **same keys** ke saath jo `clinic_data.json` mein hain (`clinic`, `providers`, `services`, `faq`, `emergency_protocol`). Is tarah `clinic_config.py` ke functions bina badle chalenge. `CONFIG_PATH` ko env var `DATA_FILE` se lo.
- **services → job types:** AC repair, AC gas refill, AC installation, AC service (cleaning), geyser repair, plumbing leak, electrical wiring, inverter/UPS check. Har ek mein duration, `price_min` / `price_max` (PKR), `provider_ids`, `approval_required` (installation, commercial = true).
- **providers → technicians:** Bilal (AC, F/G sectors), Usman (plumbing), Kashif (electrical), etc. `working_days` wahi field.
- **`booking_engine.py` mein do functions badlo:**
  - `check_emergency_protocol`: keywords → gas smell, sparking, smoke, burning smell, shock, water on DB/board. Jawab: switch na chhedein, khirkiyan kholein, bahar niklein, gas/rescue emergency line.
  - `check_contraindications` → nayi logic "needs owner approval": `approval_required` job, estimate > PKR 15,000, area service zone se bahar, commercial site.
- `extract_intent` prompt: intents mein `"question"` add karo, aur fields `area`, `issue`, `caller_phone`. Keyword map (`clinic_config.get_service`): ac, cooling, gas, geyser, leak, wiring.
- FAQ list: 15 seed FAQs (timings, visit charges, areas, warranty, payment methods).

Done jab: laptop pe `python poc.py ac_not_cooling.wav` sahi job type aur area nikaale, aur `conversation_loop.py` se ek booking `talkse.appointments` mein aa jaye.

4 Talkse web call: browser se baat, server pe pipeline 3–3.5 hours A

Problem: abhi Talkse sirf laptop ke mic ya WAV files se chalta hai, aur `conversation_loop.py` top pe `audio_playback` import karta hai jo `sounddevice` maangta hai. Docker mein yeh crash karega. Is liye server ka code in files ko import **nahi** karega.

- **`call_engine.py`**: `run_conversation` ke loop ki body ko ek function mein nikaalo:

  ```
  def start_call(caller_phone) -> (call_id, greeting_text)
  def handle_turn(call_id, audio_bytes, filename) -> {
      "transcript", "reply_text", "status",          # collecting | done | waiting_human | emergency
      "escalation_id" (optional) }
  ```

  State `talkse.calls.state_json` mein save ho (memory mein nahi), taake container restart pe call na tootay. Andar wahi existing calls: `transcribe` → `check_emergency_protocol` → `extract_intent` → approval check → `merge_state` → `next_missing_field` / `book_appointment`. `transcribe` ko file path ki bajaye bytes lene do (Groq `webm` accept karta hai).
- **`server.py`** (FastAPI):

  ```
  POST /api/calls                    → start, greeting text + audio (base64)
  POST /api/calls/{id}/turn          → multipart audio → handle_turn → reply text + audio
  GET  /api/calls/{id}/status        → waiting_human ke dauran har 2 sec poll
  GET  /                             → static/call.html
  ```

- **`static/call.html`** (plain HTML + JS, React nahi): company logo, bara **"Hold to talk"** button. `MediaRecorder` se `audio/webm` record, button chhorne pe upload, jawab ka audio `new Audio("data:audio/mpeg;base64,...")` se play. Neeche chat bubbles mein transcript. "Waiting for owner..." state pe spinner.
- **`requirements-server.txt`**: `fastapi uvicorn python-multipart groq google-genai deepgram-sdk psycopg2-binary python-dateutil python-dotenv`. `sounddevice`, `numpy`, `webrtcvad` nahi.
- Deepgram ka audio har jawab ke liye. Greeting aur "ek minute, owner se confirm karta hoon" jaise fixed jumlay ek dafa banao aur cache karo (README ka apna idea).
- Har Groq/Deepgram call pe existing retry wrappers (`safe_*_with_retry`) ka logic `call_engine.py` mein copy karo.

Mic sirf HTTPS pe chalta hai. Local testing `http://localhost:8000` pe theek hai. Live test Alternate Clouds ke HTTPS link pe karo, laptop aur phone dono se.

Done jab: live URL pe "AC not cooling, F-10, Thursday 4 PM, mera naam Sara" bolo → AI 3–4 turns mein booking confirm kare → `talkse.appointments` mein row. Har turn ka jawab ~4 sec se kam.

5 FlowCRM Lite: login hatao, 6 pages rakho 2 hours B

Clerk ko delete nahi karna. `AUTH_MODE=demo` switch lagana hai, taake baad mein real login wapas aa sake.

FILE	CHANGE
`apps/api/src/middleware/auth.ts`	Demo mode mein `requireAuth()` : Clerk skip, `DEMO_USER_ID` aur `DEMO_ORG_ID` env se, role ORG_OWNER. `requireAuthenticatedClerkSession()` seedha `next()` .
`apps/api/src/modules/auth/auth.controller.ts`	`getMe` : `getAuth(req)` ki jagah demo user ka clerkId (`DEMO_CLERK_ID`).
`apps/api/src/app.ts`	Demo mode mein Clerk webhook mount na karo. (`clerkMiddleware()` pehle se key na hone pe skip hota hai.)
`apps/web/src/app/layout.tsx`	`<ClerkProvider>` sirf jab `NEXT_PUBLIC_AUTH_MODE !== "demo"` .
`apps/web/src/middleware.ts`	Demo mode mein har request pass.
`use-api-context.ts` , `use-me.ts` , `RequireOrganization.tsx` , `onboarding/.../page.tsx`	Ek chhota `useDemoAuth()` hook: `{ isLoaded: true, isSignedIn: true, getToken: async () => null }` . Demo mode mein Clerk ke `useAuth` ki jagah yeh.
`Topbar.tsx`	`<UserButton>` ki jagah "Ahmed · Owner" avatar.
`login` , `register` , `forgot-password` pages	Demo mode mein `redirect("/dashboard")` .
`Sidebar.tsx`	Sirf Dashboard, Leads, Contacts, Pipeline, Tasks, Reports. Baaki hide (code rehta hai).
`apps/web/.env.production` (new)	`NEXT_PUBLIC_API_URL=https://flowcrm-api-app.alternatefutures.ai/api/v1` aur `NEXT_PUBLIC_AUTH_MODE=demo` . **Next.js yeh build ke waqt padhta hai**, is liye file commit karo (secret nahi hai), phir Rebuild.

AI service, Resend aur Supabase deploy nahi honge. API inke bina start hoti hai: yeh sirf tab call hote hain jab koi "score" ya "email" dabaye.

Done jab: live FlowCRM web link kholo → bina login seedha Dashboard → seeded leads, deals aur Reports nazar aayein.

6 `crm_sync.py` : har call FlowCRM mein 1–1.5 hours B

Talkse ke andar ek Python file jo FlowCRM ki `public` tables mein seedha likhti hai (koi naya API route nahi).

```
upsert_contact(phone, name)                → "Contact"      (phone se dhoondo, warna banao)
create_lead(contact_id, summary, score)    → "Lead"         source='Phone call (Talkse AI)',
notes=summary+transcript
create_deal(contact_id, title, value)      → "Deal"         stage='MEETING'  e.g. "AC repair · Sara ·
Thu 4 PM"
create_task(title, priority, lead_id)      → "Task"         priority='HIGH' for callbacks / hazards
log_activity(type, message)                → "Activity"     CALL_LOGGED, MEETING_SCHEDULED, NOTE_ADDED
```

- Table names **double quotes** mein (`"Contact"`), kyun ke Prisma ne capital letters ke saath banaye hain.
- `id` Python banaye (e.g. `"tk" + uuid4().hex`). `"updatedAt"` ka default nahi, har insert mein `now()` do. `"organizationId"` = `CRM_ORG_ID` , assignee = `CRM_OWNER_USER_ID` .
- Lead score: Talkse khud do — booked = 90, approval pending = 70, sirf sawaal = 40.
- Kab call ho: call start → contact; booking → deal + activity; escalation/timeout/hazard → task; call end → lead (summary ke saath).
- CRM write fail ho to **call kabhi na tootay**: try/except, error log, call jaari.

Done jab: browser call se booking karo → FlowCRM Pipeline mein naya card "Meeting" column mein, Contacts mein caller, Dashboard activity feed mein "Call logged".

7 Handoff agent: Telegram pe owner se poochna 2.5 hours A

Yeh project ka dil hai. Existing code mein jahan `[WOULD TRANSFER TO HUMAN]` print hota hai, wahan ab asal handoff hoga.

1. **Escalation banao:** `handle_turn` jab approval ya unknown sawaal dekhe → `talkse.escalations` mein row (`pending`), call status `waiting_human`, caller ko fixed jumla: "Ek minute, main owner se confirm kar leta hoon."
2. **Summary:** LLM se 3 lines: kaun, kya kaam, kahan, estimate. (Groq Llama, JSON nahi, sirf text.)
3. **Telegram bhejo** (`handoff.py`, plain `httpx`, koi library nahi): `sendMessage` with `inline_keyboard`. Approval ke liye buttons: `[Approve]` `[Pehle site visit]` `[Main call karunga]`. Sawaal ke liye `force_reply` taake Ahmed jawab type kare. `callback_data` = `esc:<id>:<choice>` (64 bytes se kam).
4. **Webhook:** `POST /telegram/webhook`. `setWebhook` ek dafa live URL pe, `secret_token` ke saath. Har request pe header `X-Telegram-Bot-Api-Secret-Token` check karo. Button tap → `callback_query`; typed jawab → `message.reply_to_message.message_id` se escalation dhoondo. Status `answered`.
5. **Caller tak jawab:** `call.html` har 2 sec `/status` poll kare. Answered hone pe server owner ke faisle ko ek natural jumle mein badle ("Owner ne kaha hai pehle free site visit ho, kal 11 baje theek hai?"), audio banaye, page play kare. Status `spoken`, aur conversation wahin se aage.
6. **Timeout:** 45 sec (`HANDOFF_TIMEOUT_SECONDS`) mein jawab na aaye → "Owner aap ko 2 ghante mein call karenge" → `timed_out` → `crm_sync.create_task("Callback: …", "HIGH")`.
7. Telegram message bhi edit karo jab jawab mil jaye ("✓ Site visit chosen"), taake Ahmed ko pata ho.

Local testing: webhook ke liye public HTTPS chahiye. Phase 7 seedha Alternate Clouds pe test karo, ya local pe `getUpdates` polling se. Live pe ek hi tareeqa: webhook.

Done jab: "Office mein 3 naye AC lagwane hain" → Ahmed ke phone pe Telegram message → "Pehle site visit" tap → 5 sec mein caller ko AI ka jawab → FlowCRM mein deal. Aur jawab na dene pe 45 sec baad callback task.

8 Safety + Learning agents 1.5 hours A

- **Safety (hazard):** `check_emergency_protocol` trigger ho → booking ruk jaye, caller ko safety hidayat foran, Ahmed ko Telegram alert ("⚠ Gas smell, G-11, Sara, 0300…"), FlowCRM mein HIGH task. Call status `emergency`.
- **Learning:** jab Ahmed kisi sawaal ka text jawab de, `learning.py` LLM se FAQ draft banaye (3 tarah se poocha gaya sawaal + jawab), aur Telegram pe `[Save as FAQ]` `[Sirf is dafa]`. Save → `talkse.faqs` (`source='staff'`).
- **FAQ lookup** (Receptionist ka hissa): intent `question` pe pehle `talkse.faqs` dekho. Simple tareeqa: saare FAQ questions LLM ko do, wo matching id ya `null` lautaye. `null` → escalation (Phase 7).

Done jab: "Kya aap Gree inverter AC theek karte hain?" → Ahmed jawab type kare → caller sune → Save as FAQ → **naye call** mein wohi sawaal → foran jawab, Telegram pe kuch nahi aaya.

9 Final deploy, hardening aur testing 1.5 hours A B

SERVICE	PORT	ENVIRONMENT VARIABLES
talkse	8000	`GROQ_API_KEY` `GEMINI_API_KEY` `DEEPGRAM_API_KEY` `DATABASE_URL` `TELEGRAM_BOT_TOKEN` `TELEGRAM_OWNER_CHAT_ID` `TELEGRAM_WEBHOOK_SECRET` `PUBLIC_BASE_URL` `CRM_ORG_ID` `CRM_OWNER_USER_ID` `DATA_FILE` `HANDOFF_TIMEOUT_SECONDS`
flowcrm-api	4000	`DATABASE_URL` `DIRECT_URL` `AUTH_MODE=demo` `DEMO_USER_ID` `DEMO_CLERK_ID` `DEMO_ORG_ID` `WEB_APP_URL`
flowcrm-web	3000	Build time: `apps/web/.env.production` se `NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_AUTH_MODE`

- Har service: variables → **Rebuild** → **Deploy** → Logs tab mein startup message. Telegram `setWebhook` live Talkse URL pe.
- **Error handling checklist:** khaali/silent audio ("Maaf kijiye, dobara bol dein"), LLM ka toota JSON (existing fallback), Groq rate limit (1 retry phir "ek second"), Telegram fail (seedha callback task), CRM write fail (call jaari), do log ek saath call karein.
- **10 test scripts** (B ne Phase 6 mein likhe) live link pe chalao: 3 normal bookings, 2 approvals, 1 hazard, 2 naye sawaal, 1 timeout, 1 reschedule. Phone pe **mobile data** se bhi.
- **README dono repos mein sach likho:** kya pehle se tha, kya event mein bana, kaise chalayein, architecture diagram. Talkse ka purana README jo FastAPI/React/Telnyx claim karta hai, update karo.
- Repo, video aur screenshots mein koi key ya password nahi. `.env` commit nahi.

Done jab: 10 mein se 10 scripts live pe pass. **Feature freeze.** Is ke baad sirf bug fix.

10 Demo, video aur submission 1.5–2 hours A B

TIME	6-MINUTE FINALIST DEMO (YAHI SCRIPT VIDEO MEIN CHHOTA KARKE)
0:00	Masla: "AC company ka phone June mein bajta rehta hai, aadhi calls miss, aur AI ko akela faislay nahi karne dene chahiye."
0:40	**Scene 1**: normal booking. Bari screen pe FlowCRM Pipeline mein card aata hai.
1:40	**Scene 2**: 3 AC installation → stage pe Ahmed ka phone bajta hai → "Pehle site visit" → AI caller ko batata hai.
2:50	**Scene 3**: gas smell → foran safety hidayat + alert + HIGH task.
3:40	**Scene 4**: naya sawaal → Ahmed jawab → Save as FAQ → **agli call mein foran jawab**.
5:00	FlowCRM Reports: calls → leads → booked jobs. Kaun khareedega, kitne mein. "Inspired by YC companies (Bizzy S25, Centralcoms S26) and YC's Multiplayer AI request."

- **Video** 2–3 min, unlisted YouTube ya Drive (anyone with link).
- **Screenshots:** dono teammates Alternate Futures Discord mein; Alternate Clouds dashboard pe teeno services ka deployment status.
- **LinkedIn:** ek post kare, doosra reshare. **Khud likho**, sirf grammar tool. Hashtags: #AgentikaAIHackathon #AIAutomation #AIHackathon #BuildInPublic.
- **Submission PDF** (sections 1–5): team details; project + one-line pitch + human step kyun zaroori; links (Talkse live, FlowCRM live, dono GitHub, video, LinkedIn); screenshots; confirmations. FlowCRM bina login khulta hai, is liye judges ko password nahi chahiye, lekin likh do ke demo data hai.
- **Submit 12:15 PM tak**, 12:45 ka intezaar nahi. Closing ceremony tak teeno services chalti rahein.

Done jab: submission bhej di, saare links kisi doosre phone pe khul rahe hain, aur demo do dafa timer ke saath practice ho chuka.